Code Jam 2010 - Round 2

Analysis: Bacteria

The fact that the input is a set of rectangles is not essential to this problem. We just wanted to limit the amount of data you had to download to test your program. So let us forget about the rectangles, and instead consider any initial configuration of `n` bacteria on the grid. The challenge here is to compute the answer *fast*. A turn by turn simulation will not be good enough. We are going to aim for an `O(n)` solution instead.

Examples

One way to start attacking this problem is to look at examples. The Bacteria game is pretty fun to play around with after all! Here are some examples you can try:

- A rectangle of `H` by `W` bacteria.
- A `H` by `W` rectangle where the bacteria only lie on the four boundary edges.
- Bacteria along a "type 1" diagonal (a southwest/northeast diagonal).
- Bacteria along a "type 2" diagonal (a northwest/southeast diagonal).
- A random path where each pair of bacteria are connected either horizontally, vertically, or along a type 1 diagonal.

If you work out these examples, you should get some inspiration for the general problem. We call a type 1 diagonal higher than another if it is to the north (and hence west) of the other. If the initial configuration is a "connected piece" (we will define this more precisely later), we can find the highest type 1 diagonal containing at least one bacterium $X + Y = C$, the right-most point containing a bacterium $X = X_{max}$, and the bottom-most point containing a bacterium $Y = Y_{max}$. We claim that after one turn, the configuration will still be a single connected piece, the highest diagonal will become $X + Y = C+1$, and the max X and Y coordinates will not change. This will continue until the final second when we come down to a single point (X_{max}, Y_{max}). So the number of turns before everything disappears is $X_{max} + Y_{max} - C + 1$.

One thing we note is that the 3rd and 4th examples behave very differently. In the 3rd example, the number of bacteria decreases by 1 in each turn, while in the 4th, all the bacteria disappear immediately.

Solution

Define a graph on the bacteria. Two bacteria are neighbors if they are adjacent on the grid via a horizontal line, a vertical line, or a type 1 diagonal. We begin by finding the connected components of this graph.

This may seem like a simple definition, but it is really the key to solving the whole problem. It is quite similar to the more common graph where each grid node has 8 neighbors, one in each compass direction, except we do not consider the 2 directions along a type 2 diagonal. Remember the 3rd and 4th examples above? If we start with `n` nodes, there is just one connected component in the former, but `n` connected components in the latter.

We observe that after each turn, a connected component will give birth to a new connected component (unless it was a single point and disappears), and different components will not become connected together. See the details in the next section.

So, the answer is the maximum elimination time over all the components. For a single piece (i.e., connected component), we find the highest type 1 diagonal $X + Y = C$, as well as the maximum coordinates X_{max} and Y_{max}. The number of turns for that piece to disappear is then $X_{max} + Y_{max} - C + 1$.

Justification

Our solution depends very heavily on several key observations. If you try some examples, you should be able to convince yourself empirically that these observations just have to be true. But in the interest of completeness, we will also sketch out a more formal proof here.

In the following discussion, we will fix a configuration called the *old state*, and we will consider the *new state* obtained after one turn. When we talk about a connected piece, we will always assume the non-trivial case where there are at least 2 points in the piece.

For each bacterium in the new state, let us consider the reason why it is there. If it was in the old state, we credit the reason for its existence to the set of itself and its north and/or west neighbor. If it was not in the old state, we credit the reason for its existence only to the set of its north and west neighbors. In both cases, we call such a set the credit set for that single bacterium.

Proposition 1. If the old state is a single piece, and the highest type 1 diagonal is $X + Y = C$, then in the new state, the highest type 1 diagonal will become $X + Y = C+1$.

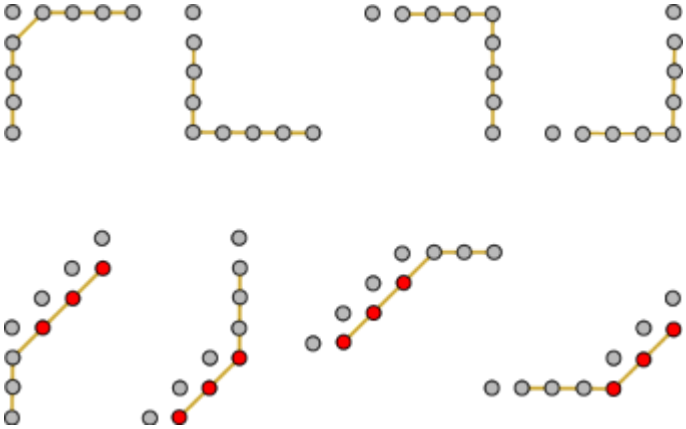
Proof. Consider a bacterium at position (X, Y) on the top diagonal of the old state. Since this is the top diagonal, that bacterium does not have a north neighbor or a west neighbor, and it will die. In particular, the diagonal $X + Y = C$ will be completely empty in the new state.
Now pick the south-most bacterium in the old state on the diagonal $X + Y = C$. Let its position be $(X, C-X)$. If there is also a bacterium at position $(X+1, C-X-1)$ in the old state, there will be a bacterium at $(X+1, C-X)$ in the new state (no matter whether it was in the old state or not). Otherwise, since $(X, C-X)$ is part of a connected piece and it's on the highest diagonal, there will be a bacterium at either $(X, C-X+1)$ or $(X+1, C-X)$ in the old state, and by our rules, this bacterium will survive to the new state. Hence, in any case, we find at least one bacterium will be on the diagonal $X + Y = C+1$ in the new state, which proves the proposition.

The following two Propositions can be justified in the same way:

Proposition 2. If the old state is a single piece, the maximum X coordinate and the maximum Y coordinate will be unchanged in the new state.

Proposition 3. If the old state is a single connected piece, the new state will be a single connected piece as well.

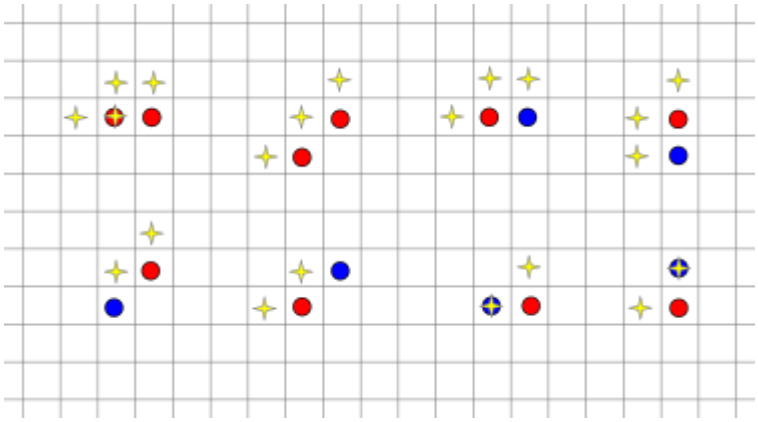
To see Proposition 3, note that if the old state is connected, then any two credit sets can be joined by some path consisting of horizontal, vertical, and type 1 diagonal segments. As the following picture shows, any possible segment from such a path will give birth to connected segments in the new state:



Finally, we are going to prove that different pieces will not be merged together in a turn.

Proposition 4. If two bacteria are neighbors in the new state, then their credit set are all from the same piece in the old state.

This is obviously true if both bacteria are also in the old state. Otherwise, it is one of the cases in the following picture. The two bacteria are the circled points. A blue point indicates that the bacterium was also in the old state, a red point indicates that the bacterium was only in the new state. Then the yellow stars represents the points in the credit sets. In any situation we either reach an impossible configuration, or get to a point where it is clear the credit sets are connected.



Code Jam 2010 - Round 2

Analysis: Elegant Diamond

On this very difficult round, even the first problem was pretty challenging. The constraints were fairly small, but it might not be clear how to even get started. After all, there are a lot of ways you can enhance a diamond!

Let's start off with a slightly simpler question: given a position (c_x, c_y), is it *possible* to enhance (i.e, extend) the given diamond into an elegant diamond centered at (c_x, c_y)? (Note that this center might be either at a number, or at a space between numbers, depending on whether the elegant diamond has even or odd side length.) The resulting diamond would have to be symmetrical about the lines $x = c_x$ and $y = c_y$. Specifically, this means that for each (x, y), the values in positions (x, y), $(2c_x - x, y)$, $(x, 2c_y - y)$, and $(2c_x - x, 2c_y - y)$ all have to be equal. If the starting diamond already has two different values in one of these quartets, there's nothing we can do to change that.

Otherwise, it turns out that the diamond always can be extended to an elegant diamond with center at (c_x, c_y). For each of the quartets above where we have one value, we have to fill out the remaining values to be equal. After that, we can just enclose everything in a diamond shape and fill all remaining squares with 0. Here's an example:

```
  1
 2 3
5 6*6
 2 3
  1
```

Let's try to extend this to an elegant diamond centered at the *. First, we fill in all quartets, and then we extend to a proper diamond by adding 0's:

```
                                0
  1            1 1              1 1
 2 3          2 3 2            2 3 2
5 6*6  -->   5 6*6 5   -->    5 6*6 5
 2 3          2 3 2            2 3 2
  1            1 1              1 1
                                0
```

Done!

This is pretty clearly the smallest extension with the given center, so all we need to do is try each possible center, and choose the one that gives the smallest possible elegant diamond.

Of course, there are a lot of possible centers, but there is no need to consider one completely outside of the bounding box of the original diamond. We can always move such a center onto the edge of the bounding box without creating any inconsistencies, and that will lead to a smaller diamond in the end.

So in summary: we need to iterate over all possible centers contained within the original diamond, check if there is an elegant extension with that center, and then take the smallest of these. As always, please take a look at some of the correct submissions to see some full implementations!

By the way, the solution presented here is O(n^4), which is fast enough for this problem. Faster solutions do exist though, and you might enjoy looking for them.

Code Jam 2010 - Round 2

Analysis: Grazing Google Goats

This problem is tough from both the implementation side and the algorithm side. In the analysis, we will begin by explaining some general concepts that show how the task is actually very similar to convex hull. We will then go into the particulars needed to build a solution and to prove that it is right. The resulting code isn't as scary as the length of the analysis makes you think.

Let us just focus on one bucket **Q** at a time. The problem asks us to choose rope lengths: i.e., for each pole position, we need to choose the radius of the circle centered at that pole. The common area will be the intersection of all these circles. It is clear that decreasing the radius of any circle will not increase the intersection. Therefore, we want the lengths to be as small as possible. Obviously, for each pole $\mathbf{P}_i$, $\mathbf{P}_i\mathbf{Q}$ is the smallest length we can pick so that the goat can still reach the bucket. Our main task is to efficiently compute the common intersection area of these circles.

From the limits of this problem, it is also clear that computing the intersection in $\Omega(\mathbf{N}^2)$ will be too slow. We are aiming for an algorithm that will, for each bucket, compute the intersection in time $O(\mathbf{N} \log \mathbf{N})$.

The intersection is a convex shape where the boundary consists of arcs of the circles. One can prove in various ways (some of the reason will be obvious below) that each circle will only contribute at most one such arc. We will mostly focus on how to compute these arcs -- to which circle does it belong, where does it stop and where does it start. Once all this is computed, it is relatively easy to get the area.

Some theoretical background

While this section is not absolutely necessary for solving the problem, it will provide some nice insights. Once we have seen the problem from a few different angles, the solution will be conceptually clear and very natural.

So, first of all, note that all the circles we consider pass through a common point **Q**. If you have not seen it before, it is now our honor to introduce to you the beautiful geometric transformation called *inversion*. It takes **Q** as the center. And each point $\mathbf{X} \neq \mathbf{Q}$ is mapped to a point **X'** where **QX** and **QX'** are in the same direction, and the lengths satisfy $|\mathbf{QX}|\,|\mathbf{QX'}| = 1$.

Here is one very nice property of inversion: Every circle passing through the center point **Q** is mapped to a line that does not pass **Q**, and vice versa. And the interior of the circle is mapped to a half-plane that does not include **Q**. For our problem, the intersection of the **N** circles is then mapped to the intersection of **N** half-planes. For more details on inversion, we refer you to the Wikipedia page on inversive geometry.

We leave it to the reader to verify that, if the intersection is not empty, then we can rotate the plane such that **Q** is above all the half-planes. And the intersection of the half-planes is bounded by something called the *lower envelope of line arrangements*. The segments of the lower envelope are exactly the images of the arcs in the circle intersection from our original problem.

Then there is a concept of *duality* in computational geometry that maps each line `y = ax + b` to the point `(a, -b)`. The lower envelope is mapped by this transformation to the upper convex hull of the set of corresponding points.

So, our problem is indeed equivalent to the convex hull problem and the lower envelope of line arrangements problem. Both subjects are well studied and there are simple O(**N** log **N**) algorithms for both.

Out of these three settings, perhaps the algorithm for line arrangement is the simplest to visualize: Sort the lines by their slopes, and add them one by one. In each step, the existing lower envelope will be cut by the new line in two parts. The time for sorting is O(**N** log **N**), and the amortized complexity for the rest is O(**N**).

Solution to our problem

The previous section suggests a couple approaches that begin with explicitly doing an inversion about **Q**. In this section, we will discuss a direct solution to the problem, where we do not need to view the problem using the transformations from the previous section. Note that it will be in some ways equivalent to the above algorithm we introduced for line arrangements.

Let's look from **Q** along a straight ray. The intersection of this ray with each circle will be either just **Q**, or a segment between **Q** and the second intersection point of this ray with the boundary of the circle. That means that in order to build the area required in the problem statement, we need to find the circle "closest" to **Q** in each direction from it. Let's denote each direction by its polar angle, which is defined up to a multiple of 2π.

We start by considering just one circle. Suppose the polar angle of a ray that goes from **Q** towards the center of that circle is φ. When the polar angle of a ray goes from $\varphi-\pi/2$ towards φ, the distance from **Q** to the second intersection point with that ray increases from 0 to the diameter of the circle. When the polar angle continues from φ towards $\varphi+\pi/2$, the distance decreases back to 0. When the polar angle changes from $\varphi+\pi/2$ to $\varphi+3\pi/2$ (the latter is the same as $\varphi-\pi/2$, where we started), the intersection of the ray with the circle is just **Q**.

Now consider two circles, one with center at polar angle φ (remember, all polar angles discussed here are relative to **Q**) , another one with center at polar angle ξ, and suppose $0 < \varphi - \xi < \pi$. Then we'll see the following as we turn the ray originating from **Q**: starting from polar angle $\xi-\pi/2$ until $\varphi-\pi/2$ the ray will intersect only the second circle; from $\varphi-\pi/2$ the ray will intersect both circles, but the intersection point with the first circle will be closer to **Q** until the circles intersect; after the intersection and until $\xi+\pi/2$ the ray will still intersect both circles but the second is now closer; and from $\xi+\pi/2$ until $\varphi+\pi/2$ the ray will only intersect the first circle.

The important thing about the two circles case discussed above is that in the range of polar angles where the ray intersects both circles (and that's exactly the range we care about in finding the intersection area). The situation is very simple: before the intersection point one circle is closer to **Q**, after the intersection point another circle is closer to **Q**.

Now suppose we have many circles. First, we find the polar angle of the center of each circle, and thus also find the range of polar angles where a ray going from **Q** intersects each of the circles. We can then intersect all those ranges of polar angles to obtain a small range $[\alpha, \beta]$ of polar angles where the ray would intersect all circles. If this range is empty, then we already know there's no intersection.

Now let's start adding circles one by one, starting from the one with the smallest polar angle. You might ask, what does "smallest" mean when we're on a circle? Luckily, here we have less than full circle: since all circles intersect all rays in $[\alpha, \beta]$ range, the polar angles of all centers are between $\beta-\pi/2$ and $\alpha+\pi/2$. That leaves us a segment of length less than π where we have a definite order.

As we're adding the circles, we'll maintain which circle is closest for each angle from $[\alpha, \beta]$. For example, after adding the first circle it will be the closest for the entire $[\alpha, \beta]$ range. Now we add

the second circle. Let's say the polar angle of the intersection point between two circles is γ. If you look carefully at the above analysis, you'll find that:

- When γ is less than α, the first circle is still the closest for the entire [α, β] .
- When γ is between α and β, the second circle is the closest for [α, γ], and the first circle is the closest for [γ, β].
- When γ is more than β, the second circle is now closest for the entire [α, β] .

Here we rely on the fact that we process circles in increasing order of the polar angle of their center.

Now look at the general case. Suppose before adding circle number i we have the following picture: on [α, γ_1] circle j_1 is the closest; on [γ_1, γ_2] circle j_2 is the closest; ...; on [γ_{k-1}, β] circle j_k is the closest.

Consider the polar angle δ of the intersection point between circle i and circle j_1. There are 3 ways it could compare with the range where j_1 is the closest:

- When δ is less than α, it means that circle j_1 is closer than circle i on the entire [α β] segment, and we can just stop processing circle i since it doesn't affect the answer.
- When δ is between α and γ_1, we now have that circle i is the closest on [α, δ], and circle j_1 is the closest on [δ, γ_1]. After doing this change, we can also stop processing circle i since circle j_1 will be closer to the center all the remaining way.
- Finally, when δ is more than γ_1, we can forget about circle j_1 since circle i is closer than it on the entire [α, γ_1] segment. In that case we continue processing circle i by comparing it with circle j_2, and so on.

That's it! After we do this processing for all circles, we know the contour of the intersection figure, and we continue by computing its area.

The above algorithm requires a data structure that maintains a list of arcs with integer tags, allowing us to change the first arc, remove the first arc, and add a new first arc. The simplest way to get this data structure is to just use a stack where the first ac would be on the top, and the last one would be on the bottom.

The running time for the above algorithm can be estimated using amortized analysis as follows. When processing each circle, we do at most two "push to stack" operations, and one or more "pop from stack" operations. That means the total number of push operations is O(**N**), and since the number of pop operations can't be greater than the number of push operations (there'd be nothing to pop!), it's also O(**N**), giving us the total runtime of O(**N**). However, we must also remember the step where we sort all circles by the polar angle of their center, so overall runtime is O(**N** log **N**).

We've so far avoided discussing the low-level computational geometry needed for this problem. In the above solution we required two geometric routines: find the polar angle of the other intersection point of two circles which have the first intersection point at origin; and find the area of a figure that is bounded by arcs.

The first routine is straightforward if you can already intersect circles (a useful thing to know how to do!); alternatively, you could derive the formulas for the polar angle directly. The second one is slightly more tricky: first, we split the figure into "rounded triangles" with rays going from **Q** and having polar angles γ_1, γ_2, ..., γ_{k-1}. Each rounded triangle has two straight sides (one of those might have zero length) and one circular side. We can then split the rounded triangle into the corresponding triangle plus the round part (a sliced off part of a circle). You have probably seen how to compute the area of a triangle before - one good approach is the "shoelace formula". To

calculate the rounded area, it helps to start with a "pie slice" from the center, whose area is a simple fraction of the total circle area, and then add or subtract triangle areas.

Of course there is a lot to do here, but that's why we made it Problem #4!

Code Jam 2010 - Round 2

Analysis: World Cup 2010

Reformulation

In the analysis, we will turn the picture upside-down. This conforms to the usual convention that the root of a binary tree is drawn on top. This is actually easy to do implementation-wise. We just read the input numbers in reverse order; the element at position 0 is the root, i.e., the final match, and for any node `i`, its two children are indexed at `2i+1` and `2i+2`. In our particular solution, we include nodes for the teams as well as for the matches, so our graph is a rooted complete binary tree, where all the internal nodes are matches, and all the leaves are the teams.

Perhaps it is clear that the solution will be dynamic programming on the tree. There are many ways to do it, but some can be much more complicated than others. Let us introduce one solution that is very simple. It is based on the following reformulation of our problem.

Let us color a node yellow if we are going to buy the ticket for the corresponding match. For each leaf (a team), the path to it from the root has `P` internal nodes. We call a plan good if no matter what the outcomes of the matches are, we will never miss more than `M[i]` matches for team `i`. We have

> (*) A plan is good if and only if for every team `i`, the path from the root to the leaf corresponding to team i has at least `P-M[i]` yellow nodes.

This is indeed a conceptual leap in solving the problem. For in the new form, we no longer need to worry about the outcome of any particular match. And once it is stated, it is very easy to justify. We leave the justification to the readers.

Simple solutions for small inputs and large inputs both follow easily from this reformulation.

Small dataset

For the small dataset the best plan must be *upwards closed*, i.e., if a node is yellow, all the nodes on the path from the root to it must all be yellow. One can prove this by using (*) and noting that all the prices are the same, and you can always get a cheaper plan if you push some yellow node upwards. So, one solution for the small dataset looks like this: For each team `i`, we find the path from root to it, and color the topmost `P-M[i]` nodes yellow. It can be done in linear time by a simple modification of depth first search.

The justification and the implementation details are a nice piece of dessert. We leave them to the readers.

Large dataset

We use dynamic programming on the tree. Let us just define the subproblems clearly. For any node `a` and any number `b` between 0 and `P`, let `P(a, b)` be the problem

> If there are `b` yellow nodes on the path from the root to `a` (not including `a`), what is the cheapest way to color the nodes in the sub-tree rooted at a, such that all the leaves in this sub-tree satisfy the condition in (*)?

And we can use a impossibly big answer to indicate that the condition can not be satisfied.

Once the DP is set up, the code follows easily. For each internal node there are just two choices, color it yellow or not. We provide an excerpt from one of the judges' solutions:

```
i64 A[1<<11][11];  // answers for the dynamic programming.
i64 inp[1<<11];    // input, in reversed order.
int P;

// rooted at a, already buy b tickets from above.
i64 dp(int a, int b) {
  i64& r=A[a][b];
  if(r>=0) return r;
  if(a>=((1<<P)-1)) {r=(b>=(P-inp[a]))?0:1LL<<40; return r;}
  r=std::min(
      inp[a]+dp(2*a+1,b+1)+dp(2*a+2,b+1),
      dp(2*a+1,b)+dp(2*a+2,b));
  return r;
}

int main() {
  int T;
  cin>>T;
  for (int cs=1; cs<=T; ++cs) {
    cin>>P; int M=(2<<P)-1;
    for (int i=0; i<M; i++) cin>>inp[M-1-i];
    memset(A, -1, sizeof(A));
    cout << "Case #" << cs << ": " << dp(0, 0) <<endl;
  }
  return 0;
}
```